

WEEK 08 · DOM, EVENTS, DEBUGGING

THE PAGE STARTS ANSWERING

Last week the language. This week it touches the document — and everything that was theory becomes something you can see going wrong.

WHERE WE ARE

Three static pages and an array that cannot reach them.

AFTER TODAY YOU CAN

Find, build, change and remove parts of a live page, react to a user, and find a bug on purpose rather than by luck.

WHAT THIS UNLOCKS

Your list renders from data, adds and removes — and week 9 gives that shape a name.

*“The DOM is a tree-like structure created by the browser that represents the HTML elements **as JavaScript objects.**”*

LESSON 9, KEPT VERBATIM – THE LAST FOUR WORDS ARE THE ONES DOING THE WORK

YOUR THREE PAGES CANNOT CHANGE

WHAT YOU BUILT IN WEEK 7

Three items, written into `index.html` by hand. They look right.

WHAT YOU ALSO HAVE

An array of items in a module, with the same three things in it.

THE PROBLEM

They are two copies of the same truth, and nothing keeps them together. Add a fourth item to the array and the page does not care.

A page that cannot be changed by code is a picture of an application.

THREE QUESTIONS, AND THE WHOLE DOM API ANSWERS THEM

WHERE IS IT?

`querySelector`, `closest`,
`traversal`. Acts 2–3.

HOW DO I CHANGE IT?

`createElement`, `textContent`,
`append`, `classList`. Acts 4–6.

HOW DO I HEAR THE USER?

`addEventListener`, the event
object, delegation. Acts 7–8.

► Then act 9, which is the half you will still be using in ten years: **HOW TO FIND A BUG ON PURPOSE.**

THE FILE AND THE DOCUMENT ARE TWO DIFFERENT THINGS

left: what the server sent · right: the tree, walked right now

WHAT THE SERVER SENT · WHAT THE DOCUMENT IS NOW

Reading list

- Eloquent JavaScript

add a row

set an attribute

change the heading text

insert a comment node

reset

THE FILE, AS SERVED (FETCHED ONCE, NEVER RE-READ)

```
<div id="stage">
  <h4>Reading list</h4>
  <ul id="rows">
    <li>Eloquent JavaScript</li>
  </ul>
</div>
```

THE TREE, RIGHT NOW

```
<div> id="stage"
  #text ·whitespace·
  <h4>
    #text "Reading list"
    #text ·whitespace·
    <ul> id="rows"
      #text ·whitespace·
      <li>
        #text "Eloquent JavaScript"
        #text ·whitespace·
      #text ·whitespace·
    </li>
  </ul>
</div>
```

nodes	elements	text nodes	of them whitespace only
11	4	7	5

stage.firstChild is **#text (whitespace)** · stage.firstElementChild is **<h4>**. The indentation in the file is a text node, and it is why the first one surprises people.

► ADD THREE ROWS, THEN ASK WHAT **CTRL+U** WILL SHOW. Take a vote before you press it.

EVERY NODE IS A NODE. NOT EVERY NODE IS AN ELEMENT.

```
1 stage.childNodes      // NodeList(5) - text, h4, text, ul, text
2 stage.children        // HTMLCollection(2) - h4, ul
3 stage.childElementCount // 2
4
5 stage.firstChild      // #text - the indentation in your HTML file
6 stage.firstElementChild // <h4> - what you meant
7
8 // the whitespace between tags is a REAL node, and it is why
9 // firstChild surprises people the first time
```

The indentation you write for humans is data the browser keeps.

THE SCRIPT THAT RAN BEFORE THE PAGE EXISTED

THE BUG

```
1 <head>
2   <script>
3     document.querySelector('#list').append(row);
4   </script>
5 </head>
6 <body>
7   <ul id="list"></ul>
8   <!-- TypeError: ... of null -->
9 </body>
```

THE FIX

```
1 <head>
2   <script type="module" src="js/app.js"></script>
3 </head>
4 <body>
5   <ul id="list"></ul>
6 </body>
7
8 <!-- a module is deferred: it runs after the
9      document is parsed. So is defer. -->
```

YOU ALREADY KNOW THE SELECTOR LANGUAGE

the week-2 selectors, handed to JavaScriptreloadopen

ONE SELECTOR, FOUR QUESTIONS THE PAGE ANSWERS FOR YOU

selector

.card p .card > p p.meta #first-card article .sale article:first-of-type h4 .card:not(.wide) .cart button.buy #

THE MARKUP BEING SEARCHED

Coffee grinder

Burr, 40 steps, quiet.

In stock · 89

Kettle

Gooseneck, variable temperature.

Backorder · 120

Scale

0.1 g, two-year battery.

In stock · 45

WHAT THE BROWSER ANSWERED

`querySelectorAll` → `NodeList(6)`

`querySelector` → `p.match.first` – the first match in document order, highlighted in yellow

Matched: `p.match.first` · `p.meta.match` · `p.match` · `p.meta.match` · `p.match` · `p.meta.match`

Both of these were taken before you pressed anything: `getElementsByClassName('card')` → `3` · `querySelectorAll('.card')` → `3`. Press "add a card" and watch which of the two numbers changes on its own.

► Type `.card p`, then `.card > p`. **LET THE ROOM EXPLAIN THE DIFFERENCE FROM THE HIGHLIGHT COUNT ALONE.**

null IS AN ANSWER, NOT A FAILURE

```
1 const box = document.querySelector('#totl'); // typo
2 box.textContent = '9'; // TypeError: Cannot set properties of null
3
4 // READ THE MESSAGE AS TWO HALVES:
5 //   what was null → box, the result of the selector
6 //   what you did with it → setting "textContent"
7
8 // so the bug is one line ABOVE the line in the trace.
9 // a selector that matched nothing does not complain.
```

The error is where it broke. The bug is where the wrong value was made.



WHY `getElementById` IS STILL HERE

What it is The original lookup, from 1998. One id, one element, no selector parsing.

Key trait Marginally faster, and **it cannot be given a wrong selector** — there is no syntax to get wrong.

Use cases Reading it in code that predates you.
Writing it when the thing genuinely has an id.

Tips Use `querySelector` by default: one function, every case, the language you already know.

THE GOTCHA WORTH KNOWING

`getElementById('list')` takes the **id**.

`querySelector('#list')` takes a **selector**.

Passing `'#list'` to the first one silently finds nothing, forever.

ONE OF THESE KEEPS LOOKING. THE OTHER TOOK A PHOTOGRAPH.

```
1  const live = document.getElementsByClassName('card'); // HTMLCollection
2  const snap = document.querySelectorAll('.card');      // NodeList
3
4  addOneMoreCard();
5
6  live.length    // 4  - it re-answers the question every time you ask
7  snap.length    // 3  - it was true when you asked, and never again
```

TRAVERSING FROM AN ELEMENT

```
parentElement · children · nextElementSibling ·
closest(sel) · matches(sel)
```

THE ONE YOU WILL USE TODAY

`closest()` walks **up** until something matches. Act 7 is built on it.

BUILDING A ROW, ONE OBJECT AT A TIME

```
1  const li = document.createElement('li');
2  li.dataset.id = item.id;           // <li data-id="a1">
3
4  const title = document.createElement('span');
5  title.className = 'item-title';
6  title.textContent = item.title;    // TEXT. never parsed as markup.
7
8  const remove = document.createElement('button');
9  remove.type = 'button';
10 remove.className = 'remove';
11 remove.append('הסר');
12 li.append(title, remove);          // append takes several at once
```

Nothing here is on the page yet.
It exists, and it is not attached.

THREE WAYS TO ADD A ROW. ONE OF THEM KILLS YOUR LISTENERS.

same result on screen, three different documents underneath

reloadopen

SAME RESULT ON SCREEN · THREE DIFFERENT DOCUMENTS UNDERNEATH

add one row to each listtime 200 insertreset

```
list.innerHTML += '<li>_</li>'
```

Reads the whole list back as text, appends to that string, and hands the result to the parser.

row 1 — the one with the handler

rows1

```
list.insertAdjacentHTML('beforeend', ...)
```

Parses the new fragment only, and puts it where you said. Existing children are not touched.

row 1 — the one with the handler

rows1

```
list.append(document.createElement(...))
```

Builds one element object and attaches it. No HTML is parsed at all, so no string can turn into markup.

row 1 — the one with the handler

rows1

Press **add one row**. Before each insert a click handler is attached to the first row of every list, and afterwards the page dispatches a real click on it.

► **BEFORE PRESSING: ASK WHICH COLUMN WILL BREAK.** Most of the room votes for the third.

12

THE LOOP THAT DESTROYS WHAT IT JUST BUILT

THE BUG

```
1 for (const item of items) {  
2   list.innerHTML += `<li>${item.title}</li>`;  
3 }  
4  
5 // every pass re-reads the whole list as a string,  
6 // re-parses it, and throws away every element  
7 // that was there – with its listeners.
```

THE FIX

```
1 list.replaceChildren();  
2 for (const item of items) {  
3   list.append(itemRow(item));  
4 }  
5  
6 // or, when it must be a string:  
7 list.insertAdjacentHTML('beforeend', html);
```

The rows look identical. They are different objects, and only one set has your listeners.

REMOVING, AND PUTTING THINGS IN THE RIGHT PLACE

```
1 row.remove();           // the element removes itself
2
3 list.replaceChildren();  // empty the container
4
5 list.prepend(newest);    // first child instead of last
6
7 target.before(banner);   // as a sibling, in front
8 target.after(banner);    // as a sibling, behind
9
10 list.insertAdjacentHTML('afterbegin', html); // 4 positions, one call
```

THE OLD WAY YOU WILL MEET IN OLD CODE

`parent.removeChild(child)` and `parent.appendChild(one)`. They work. They need the parent, and they take one node.

THREE PROPERTIES, AND THEY ARE NOT INTERCHANGEABLE

`textContent`

Reads and writes **text**. Never parses. Fast, because it does not have to lay anything out.

`innerHTML`

Hands the string to the **HTML parser**. Necessary sometimes. Never for text a user typed.

`innerText`

Reads what is **rendered** — skips hidden elements, respects CSS. To know that, it must lay the page out first.

► The performance difference is a footnote. **THE NEXT SLIDE IS THE REASON.**

THE REASON, AND IT IS NOT THE SPEED

one string, three properties, and a counter on **window** reload open

ONE STRING · THREE PROPERTIES · ONE OF THEM EXECUTES IT

the "review" a user typed assign to all three

plain text

markup, not script

a script tag (does nothing!)

the one that actually runs

and it is not about images

box.textContent = s

Great grinder, 5 stars

child elements created 0

shows the characters you typed **yes**

box.innerHTML = s

Great grinder, 5 stars

child elements created 0

shows the characters you typed **yes**

box.innerText = s

Great grinder, 5 stars

child elements created 0

shows the characters you typed **yes**

Nothing ran. window.__ran is 0.

INNERTEXT IS NOT TEXTCONTENT, AND THE DIFFERENCE IS CSS

The kettle is LOUD.

PROPERTY	WHAT IT RETURNED	LENGTH
textContent	The kettle is loud – internal note, do not ship.	48
innerText	The kettle is LOUD.	19
innerHTML	The kettle is loud – internal note, do not ship.	101

textContent reads the tree. innerText reads the rendered result, so it drops the hidden span, and to know what is hidden it has to lay the page out first. That is the performance footnote — and it is a footnote.

► **PRESS THE `<SCRIPT>` PRESET FIRST AND LET THEM RELAX.** Then press the next one.

THE RULE, IN ONE LINE

THE BUG

```
1 const review = form.review.value;
2 box.innerHTML = `<p>${review}</p>`;
3
4 // 
6 // runs with the logged-in user's session.
```

THE FIX

```
1 const p = document.createElement('p');
2 p.textContent = review;
3 box.replaceChildren(p);
4
5 // the structure comes from createElement,
6 // the text comes from the user. never mixed.
```

Text a person typed never reaches the HTML parser. That is the whole rule.

LIVE CODING · 6 MINUTES

RENDER THE LIST FROM THE ARRAY

- 1 Open the week-7 page. Delete every `<li>` from the markup. The list is empty and the page still loads.
- 2 Write `itemRow(item)` — four `createElement` calls and a `return`.
- 3 Write `render()` — `replaceChildren()`, a loop, `append`.
- 4 Add a fourth item to the array.
Refresh.

► That last step is act 1's problem, gone. **SAY SO OUT LOUD WHEN IT HAPPENS.**

STATE GOES IN CLASSES. VALUES GO IN `style`.

```
1 row.classList.add('selected');
2 row.classList.remove('selected');
3 row.classList.toggle('selected'); // and toggle(name, force)
4 row.classList.contains('selected'); // true / false
5
6 // a computed value has nowhere else to go:
7 bar.style.width = `${percent}%`;
8
9 // this is not a value you computed. it is a decision.
10 badge.style.backgroundColor = 'gold'; // now you have a problem
```

A class is a decision the stylesheet can argue with. An inline style is a fact it cannot.

WHAT AN INLINE STYLE TAKES AWAY

add a class, then write one inline style, then add a class again

reload

open

A CLASS IS A DECISION · AN INLINE STYLE IS A FACT YOU CANNOT ARGUE WITH

A · STYLING THE BADGE

in stock

classList.add('ok')

classList.add('danger')

classList.toggle('danger')

style.backgroundColor = 'gold'

now try a class again

reset

class attribute	(none)
classList.contains("danger")	false
style.backgroundColor (inline only)	(empty string)
getComputedStyle → background	rgb(241, 245, 249)
dataset.sku · dataset.unitPrice	GR-40 · 89

Press the class buttons first. Then press the inline one, then **now try a class again**.

[a relative link, for the third pair](#)

B · THE ATTRIBUTE IS NOT THE PROPERTY

the markup says value="pencil" and no checked

quantity note pencil

☐ gift wrap

READ AS	PROPERTY	GETATTRIBUTE
input.value	"pencil"	"pencil"
checkbox.checked	false	null
a.href	"http://127.0.0.1:50295/weeks/week-08-dom-debug/demos/01-live-tree/index.html"	"../01-live-tree/index.html"
badge.dataset.sku	"GR-40"	"GR-40"

Yellow means the two disagree. The attribute is the **HTML file**; the property is the **element right now**. A form is read through properties — always.

► The class **IS** added — the class attribute proves it — and the colour does not move.

THE ATTRIBUTE IS THE FILE. THE PROPERTY IS THE ELEMENT.

```
1 // markup: <input id="q" value="pencil"> ... then a user types "notebook"
2 q.value // "notebook" ← what is on screen
3 q.getAttribute('value') // "pencil" ← what the file said
4
5 gift.checked // true
6 gift.getAttribute('checked') // null - it never changed
7
8 link.href // "http://localhost:5173/about.html" (resolved)
9 link.getAttribute('href') // "about.html" (as written)
10
11 row.dataset.id // "a1" ← data-id="a1", both ways
12 row.dataset.unitPrice // data-unit-price, camelCased
```

A form is read through properties. Always.

LISTENING, AND THE PARENTHESES THAT RUIN IT

THE BUG

```
1 button.addEventListener('click', handleClick());
2
3 // handleClick RUNS, right now, and its
4 // RETURN VALUE – probably undefined – is
5 // registered as the listener. nothing
6 // happens when you click. nothing errors
7 // either.
```

THE FIX

```
1 button.addEventListener('click', handleClick);
2
3 button.addEventListener('click', (event) => {
4   handleClick(event, extraThing);
5 });
```

You are handing over the function itself. Parentheses call it.

THE OBJECT THE BROWSER HANDS YOU

```
1 list.addEventListener('click', (event) => {  
2   // one argument, and it is not optional reading  
3  
4   event.type           // 'click'  
5   event.target         // WHAT WAS HIT – often a span inside a button  
6   event.currentTarget   // WHO IS LISTENING – always `list` here  
7   event.key            // on a keyboard event: 'Enter', 'Escape', 'a'  
8  
9   event.preventDefault(); // do not do the browser's default thing  
10  event.stopPropagation(); // do not let anyone above hear this  
11 });
```

THE TWO THAT GET CONFUSED, AND IT IS THE BUG IN THIS WEEK'S CHALLENGE

`target` changes with every click. `currentTarget` is the element you attached to, and it never changes.

ONE CLICK, SEVEN LISTENERS, ONE ORDER

capture goes down, bubble comes back up – and the browser chose the order, not this page↻ reloadopen ↗

ONE CLICK · SEVEN LISTENERS · ONE ORDER, AND THE BROWSER CHOSE IT

☐ `stopPropagation()` in the middle box, bubble phase clear the log

OUTER · DIV#OUTER

MIDDLE · DIV#MIDDLE

INNER · DIV#INNER

Add to cart

Click the button. Then click the green area around it, then the blue area. The target column changes; the `currentTarget` column is the listener that is speaking.

WHAT FIRED, IN ORDER

#	PHASE	CURRENTTARGET	TARGET
phase 1 = capturing (going down) · 2 = at the target · 3 = bubbling (coming back up)			

► **ASK HOW MANY HANDLERS RAN.** Nobody says seven.

preventDefault AND stopPropagation ARE NOT A PAIR

preventDefault()

Stops what the **browser** would do: submit the form, follow the link, tick the box. Uses it every day.

stopPropagation()

Stops **other code** from hearing the event. Breaks the outside-click handler that closes a menu. Almost never the answer.

THE TELL

Reaching for `stopPropagation` to stop a handler firing twice means **you registered it twice**. Fix that.

They sound similar and they cancel completely different things.

A LISTENER IS ATTACHED TO AN ELEMENT, NOT TO A SELECTOR

THE BUG

```
1 for (const b of list.querySelectorAll('.remove')) {  
2   b.addEventListener('click', onRemove);  
3 }  
4  
5 // runs ONCE, over the elements that exist NOW.  
6 // a row added later has no listener.  
7 // and after render() every row is a new object.
```

THE FIX

```
1 list.addEventListener('click', (event) => {  
2   const button = event.target.closest('.remove');  
3   if (!button) return;  
4   onRemove(button.dataset.id);  
5 });  
6  
7 // one listener, attached once, to a container  
8 // that render() empties but never replaces.
```

DELEGATION, MEASURED

same array, same render, one listener in a different place

↺ reload

open ↗

SAME ARRAY · SAME RENDER · ONE LISTENER IN A DIFFERENT PLACE

add an item

re-render both

reset

☐ re-bind after every render (the usual "fix")

One listener per button

```
list.querySelectorAll('.remove')  
  .forEach(b => b.addEventListener('click', ...))
```

Burr grinder

remove

Gooseneck kettle

remove

Bench scale

remove

addEventListener calls so far 0

rows on screen 3

rows whose button is wired 0 of 3

clicks this list heard 0

One listener on the list

```
list.addEventListener('click', (e) => {  
  const b = e.target.closest('.remove'); ... })
```

Burr grinder

remove

Gooseneck kettle

remove

Bench scale

remove

addEventListener calls so far 1

rows on screen 3

rows whose button is wired all 3, by one listener

clicks this list heard 0

Both lists work right now. Press **add an item**, then try to remove the yellow row on each side.

► **"ADD AN ITEM" KILLS ONE ROW. "RE-RENDER" KILLS ALL OF THEM.** Two buttons, two different bugs.

this INSIDE A LISTENER

```
1 button.addEventListener('click', function () {
2   this;           // the button - addEventListener sets it
3   this.disabled = true;
4 });
5
6 button.addEventListener('click', () => {
7   this;           // NOT the button. an arrow has no `this` of its own,
8                   // so it is whatever `this` was where you WROTE it.
9 });
10
11 // the rule that removes the question entirely:
12 // use event.currentTarget, and never think about it again.
```

Week 7 said the call site decides.
addEventListener is a call site.

THE DEFAULT ACTION IS A PAGE LOAD

left: no preventDefault, in a frame so the demo survives it

reload

open

THE DEFAULT ACTION IS A PAGE LOAD · THE VALUE IS ALWAYS A STRING

NO PREVENTDEFAULT — IN A FRAME, SO IT CAN SURVIVE

note

type something

Save

this page has loaded 1 time(s)

location.search (empty)

the field now holds (empty — what you typed is gone)

No preventDefault anywhere in this page.

Type something and press Save. The counter goes up: the browser asked the server for that page again. Every variable that page held is gone, which is why a half-finished list disappears.

HANDLED: PREVENTDEFAULT, VALIDATE, RESPOND

item

quantity

1

Add

The page never reloads, the list is built from a JavaScript array, and a failed submit says **which field** and **why** — then puts the cursor in it.

EXPRESSION

RESULT

WHY

`typeof qty.value`

"string"

the DOM stores what was typed, as text

`qty.value === 5`

false

a string is never strictly equal to a number

`qty.value == 5`

false

loose equality converts the string first

`qty.value > 3`

false

a comparison operator converts too — which is why the bug survives testing

`Number(qty.value)`

1

the conversion you should write yourself

`Number.isInteger(Number(qty.value))`

true

what a quantity check actually needs

► TYPE SOMETHING, PRESS SAVE, AND ASK WHAT JUST HAPPENED. The answer is usually “nothing” or “it flashed”.

`input.value` IS A STRING. ALWAYS.

```
1  const qty = document.querySelector('#qty'); // <input type="number">
2  typeof qty.value // "string"
3
4  qty.value === 5 // false - a string is never === a number
5  qty.value == 5 // true - loose equality converts
6  qty.value > 3 // true ← AND THIS IS WHY THE BUG SURVIVES
7
8  qty.value + 1 // "51" - plus with a string concatenates
9  Number(qty.value) + 1 // 6
10
11 Number('') // 0, not NaN. an empty field passes "is it a number".
```

Convert once, at the boundary,
where the string comes in.

FAILING WELL IS THREE THINGS, NOT ONE

```
1 form.addEventListener('submit', (event) => {  
2   event.preventDefault();  
3  
4   if (title.trim().length < 2) {  
5     formError.textContent = 'לשם צריכים להיות לפחות שני תווים.'; // WHAT is wrong  
6     field.setAttribute('aria-invalid', 'true'); // WHICH field  
7     field.focus(); // WHERE to type  
8     return;  
9   }  
10  
11   field.setAttribute('aria-invalid', 'false'); // and CLEAR it when it passes  
12 });
```

MESSAGES THAT ARE WORTH ZERO

“Error” · “Invalid input” · “Please fill in all fields”. All true. None of them can be acted on.

LIVE CODING · 6 MINUTES

THE ADD FORM, FROM NOTHING

- 1 `form.addEventListener('submit', ...)` and `event.preventDefault()`. Press the button: nothing reloads.
- 2 Read both fields. `Number()` the second one, out loud, at the boundary.
- 3 Push into the array, `render()`. A row appears.
- 4 Now submit with an empty name — and watch it push an empty row.
Then fix it.

► **DO STEP 4 FOR REAL.** Validation that arrives after the bug is remembered; validation that arrives first is a rule they will forget.

SECOND HALF

FINDING A BUG ON PURPOSE

- 1 Reproduce**
— what sequence makes it happen, every time?
- 2 Isolate**
— which file, which function, which line?
- 3 Hypothesise**
— “I think X is Y when it should be Z.”
- 4 Verify**
— one breakpoint that makes that sentence true or false.
- 5 Fix**
— the cause, not the symptom.
- 6 Confirm**
— redo step 1, and check what is beside it.

► A hypothesis you cannot disprove is not a hypothesis. **IT IS A MOOD.**

ELEMENTS: THE DOM, EDITABLE, IN FRONT OF YOU

WHAT IT SHOWS

The **document**, not the file.
Everything JavaScript added is here.

LIVE EDIT

Double-click a class, an attribute, a text node. The page changes.
Nothing is saved — refresh and it is gone.

STYLES & COMPUTED

Which rule won, which lost with a strike-through, and the final number after everything.

► Select an element, switch to Console, type `$0`. **IT IS THE ELEMENT YOU JUST CLICKED.**

THE CONSOLE HAS FIVE MORE THINGS IN IT

each one answers exactly one question

↻ reload

open ↗

SIX CALLS, THE QUESTION EACH ONE ANSWERS, AND THE OUTPUT MIRRORED ON THE PAGE

A card with two data- attributes, for dir to look at.

console.table(items)

console.log(el) vs console.dir(el)

console.group()

console.count()

console.time()

console.trace()

MIRRORED OUTPUT — THE CONSOLE HAS THE SAME THING IN IT

Open DevTools (F12 · Cmd+Option+I), then press a button.

WHICH ONE ANSWERS WHICH QUESTION

QUESTION	CALL
what is in this array of objects?	console.table(list)
what properties does this element have?	console.dir(el)
which of these forty lines belong together?	console.group / groupEnd
how many times did this really run?	console.count(label)
how long did that take?	console.time / timeEnd
who called this function?	console.trace()

Two more that only exist in the console itself, so they cannot be on this page: \$0 is the element you last selected in the Elements panel, and \$\$('li.card') is querySelectorAll with less typing. Select something in Elements, switch to Console, type \$0.

► **CONSOLE.LOG(EL) AND CONSOLE.DIR(EL) ARE THE SAME CALL WITH DIFFERENT ANSWERS.**

SOURCES: STOPPING TIME

BREAKPOINT

Click the line number. Execution stops **before** that line, with every variable in scope readable.

CONDITIONAL

Right-click the line:

`item.id === 'a4'`. Stops on the one iteration you care about, out of forty.

WATCH

An expression re-evaluated at every pause. `guests.map(g => typeof g.seats)` is a whole diagnosis.

► **STEP OVER** runs the next line · **STEP INTO** goes inside the call · **STEP OUT** finishes it · and the **CALL STACK** panel is who is waiting for you.

READING A STACK TRACE PROPERLY

four real failures, four real traces↶ reloadopen ↷

FOUR REAL FAILURES · FOUR REAL TRACES

A paragraph with id="target", so failure 1 can be about the selector and not about the page.

1 · a selector that matched nothing

2 · a property of undefined

3 · a guard that threw, three calls deep

4 · inside an event listener

THE ERROR, AS THE CONSOLE SHOWS IT

Press a button.

THE CODE THAT RAN

—

The method, every time.

1. **Reproduce** — what exact sequence makes it happen, every time?
2. **Isolate** — which file, which function, which line?
3. **Hypothesise** — one sentence: "I think X is Y when it should be Z."
4. **Verify** — a breakpoint or one log that makes the hypothesis true or false.
5. **Fix** — change the cause, not the symptom.
6. **Confirm** — redo step 1, and check that nothing near it broke.

► FRAME 0 IS WHERE IT THREW. THE LINE TO OPEN IS THE FIRST FRAME IN A FILE YOU WROTE.

NETWORK: WHEN IT IS NOT YOUR JAVASCRIPT

STATUS

404 — the path is wrong. **200 but empty** — the path is right and the file is not what you think.

TYPE FILTER

Filter to `JS` and see whether your module was even requested. A typo in `src` is silent otherwise.

THE ONE THAT WASTES AN AFTERNOON

The file loaded — from the **cache**. Tick **Disable cache** while DevTools is open, permanently.

Before debugging what your code did with the data, check that the data arrived.

THREE QUESTIONS BEFORE YOU GO

- 1 A page has ten rows and you attached ten listeners at load. A row is added.
How many rows respond, and why?
- 2 `qty.value` is "5". Which of these is true: `=== 5`, `== 5`, `> 3`?
Why does that combination hide bugs?
- 3 You get `Cannot read properties of null (reading 'append')`.
What is null, and where is the bug relative to the line in the trace?

TODAY'S HANDS-ON, AND IT IS IN TWO HALVES

35 MINUTES · ALONE

Five planted faults in one application. Fix them, and write down **how you found each**. No AI, no search, no neighbour.

55 MINUTES · YOUR PROJECT

The list renders from the array, add and remove work, the form fails well. **Delegation is required**, not suggested.

BEFORE WEEK 9

Nothing to install. Home assignment 3 goes out today — an interactive app of your own, due before week 10.

► **THE WORKSHEET IS HALF THE MARKS OF THE FIRST HALF**, and the line that decides is “how did you test it”.